# Middleware Overview

One of the most important parts of any web application is the middleware, which we have yet to really get into.

Middleware is a type of filter that sits between the HTTP request from the client and the application. It checks and processes incoming requests before they reach the controller or outgoing responses before they are sent back to the user.

For example, you can use middleware to verify that the user of your application is authenticated. If the user is not authenticated, the middleware will redirect the user to the login page. We are going to implement this but before we do that, I want to create some simple custom logging middleware.

Think of middleware as a gatekeeper for your application. It sits between the user and the application and decides whether the user is allowed to access the application.

## Global vs Route Middleware

There are two types of middleware: global and route. Global middleware is run for every request, while route middleware is run for a specific route. I am going to show you both ways in this lesson.

## Creating Middleware

Let's create a simple logging middleware that will log the request method and URI to the log file.

To create a middleware, we can use the `make:middleware` command:

```
php artisan make:middleware LogRequest
```

This command will create a new middleware class in the `app/Http/Middleware` directory. The class has one method called `handle`. This method is called when the middleware is run. It returns `$next($request);` by default, which means that it will pass the request on to the next middleware in the chain.

Next, let's open the `app/Http/Middleware/LogRequest.php` file and just add a simple print for now:

```
public function handle(Request $request, Closure $next): Response
 {
     print('From the LogRequest middleware');
     return $next($request);
 }
```

## Registering Global Middleware

To register the middleware to run globally on all routes, we need to add it to the `$routeMiddleware` array in the `bootstrap/app.php` file. Open that file and import the middleware:

```
use App\Http\Middleware\LogRequest;
```

Then add it to the `->withMiddleware()` closure:

```
->withMiddleware(function (Middleware $middleware) {
  $middleware->append(LogRequest::class);
})
```

The append method adds the middleware to the end of the list of global middleware. If you would like to add a middleware to the beginning of the list, you should use the prepend method.

Now when you go to any route, you will see the print statement.

## Log Requests

Instead of printing, let's use the `Log` facade to log the request method and URI to the log file. Open the `app/Http/Middleware/LogRequest.php` file and replace the `handle` method with the following:

```
public function handle(Request $request, Closure $next): Response
{
    Log::info("{$request->method()} - {$request->fullUrl()}");
    return $next($request);
}
```

Now your requests will be logged to the `storage/logs/laravel.log` file. Open it up and go to the end of the file and you should see something like the following:

```
[2024-08-19 11:15:38] local.INFO: GET - http://127.0.0.1:8000
[2024-08-19 11:15:41] local.INFO: GET - http://127.0.0.1:8000/jobs
[2024-08-19 11:16:01] local.INFO: GET - http://127.0.0.1:8000/jobs/1
```

Later on, we will learn more about logging. I will show you how to create an artisan command to clear the log file.

## Assigning Middleware to Routes

So this is global middleware. Let's say you want this to only run on a specific route. You can do that by passing the middleware to the route.

First, delete or comment out the following line in the `bootstrap/app.php` file:

```
  ->withMiddleware(function (Middleware $middleware) {
        // $middleware->append(LogRequest::class);
```

```
    })
```

Now open the `routes/web.php` file and add the following import:

```
use App\Http\Middleware\LogRequest;
```

Then add the middleware to the home route:

```
Route::get('/', [HomeController::class, 'index'])->name('home')-
>middleware(LogRequest::class);
```

Now you will only see logs from the home route. This is a bit pointless, but it shows you how to use middleware.

Go ahead and remove the middleware from the home route. You can delete the `LogRequest` middleware if you want.